

UNIFIED MODELING LANGUAGE

UML

UML Diagrams

UML (Unified Modeling Language) is a standard way to visually design and represent software systems using **diagrams**.

The main purpose is to have a common **visual language** that shows how **classes, objects, relationships, and interactions** work in a system.

This makes it easier for **developers** and **teams** to understand, discuss, and plan the system before writing actual code.

UML Diagrams Importance

Visualization: UML diagrams let you see the system visually, like a **blueprint**. This makes it much easier to understand how different parts of the system are **structured**, how they **connect**, and how they **work together**.

Documentation: UML diagrams act as detailed documentation of your software design. This is very helpful when you need to **maintain** the code later or when the system needs to grow **bigger**.

Communication: UML diagrams create a common language that **designers, developers, and teams** can all understand. Everyone can look at the same diagram and discuss the design clearly without confusion.

Standardization: Since UML is a **universally accepted standard**, everyone in the software industry reads and understands these diagrams the same way. No matter where you work, **UML will be the same**.

Basic Elements

1. Class: A blueprint or template for creating objects. It has a **name**, properties (**attributes**), and actions (**methods**) that objects can perform.

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def login(self):
        pass
```

2. Interface: In Python, we use Abstract Classes to define which methods a class must have. It tells what a class should do, not how to do it.

```
from abc import ABC, abstractmethod

class Loginable(ABC):
    @abstractmethod
    def login(self):
        pass
```

Basic Elements

3. Object: A real working copy of a class created when the program runs. If Class is a blueprint, Object is the actual house built from it.

- Example: `user1 = User("Anirudh", 25)`

4. Association: A connection between two classes showing how their objects interact or relate to each other.

5. Inheritance: An "is-a" relationship where a child class inherits properties and methods from a parent class.

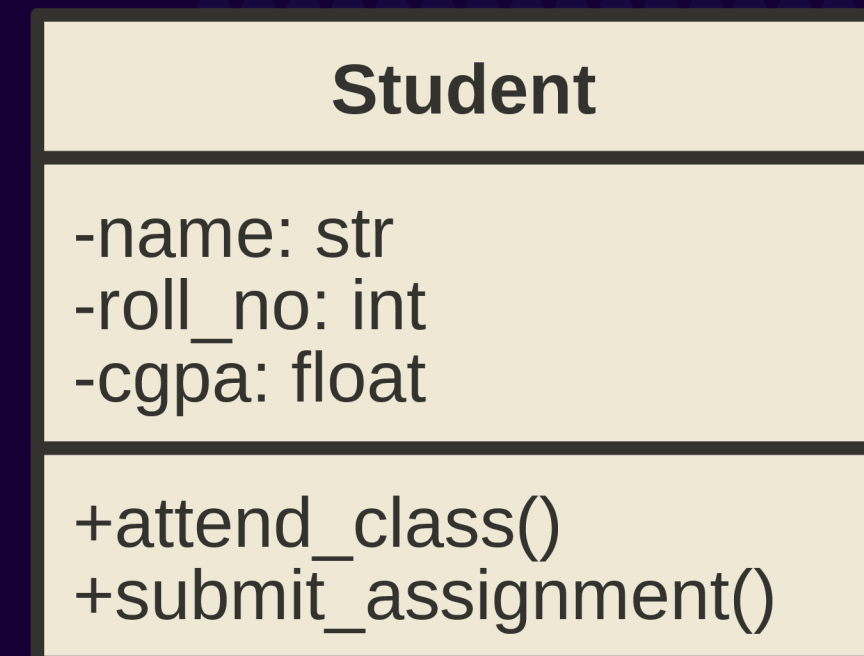
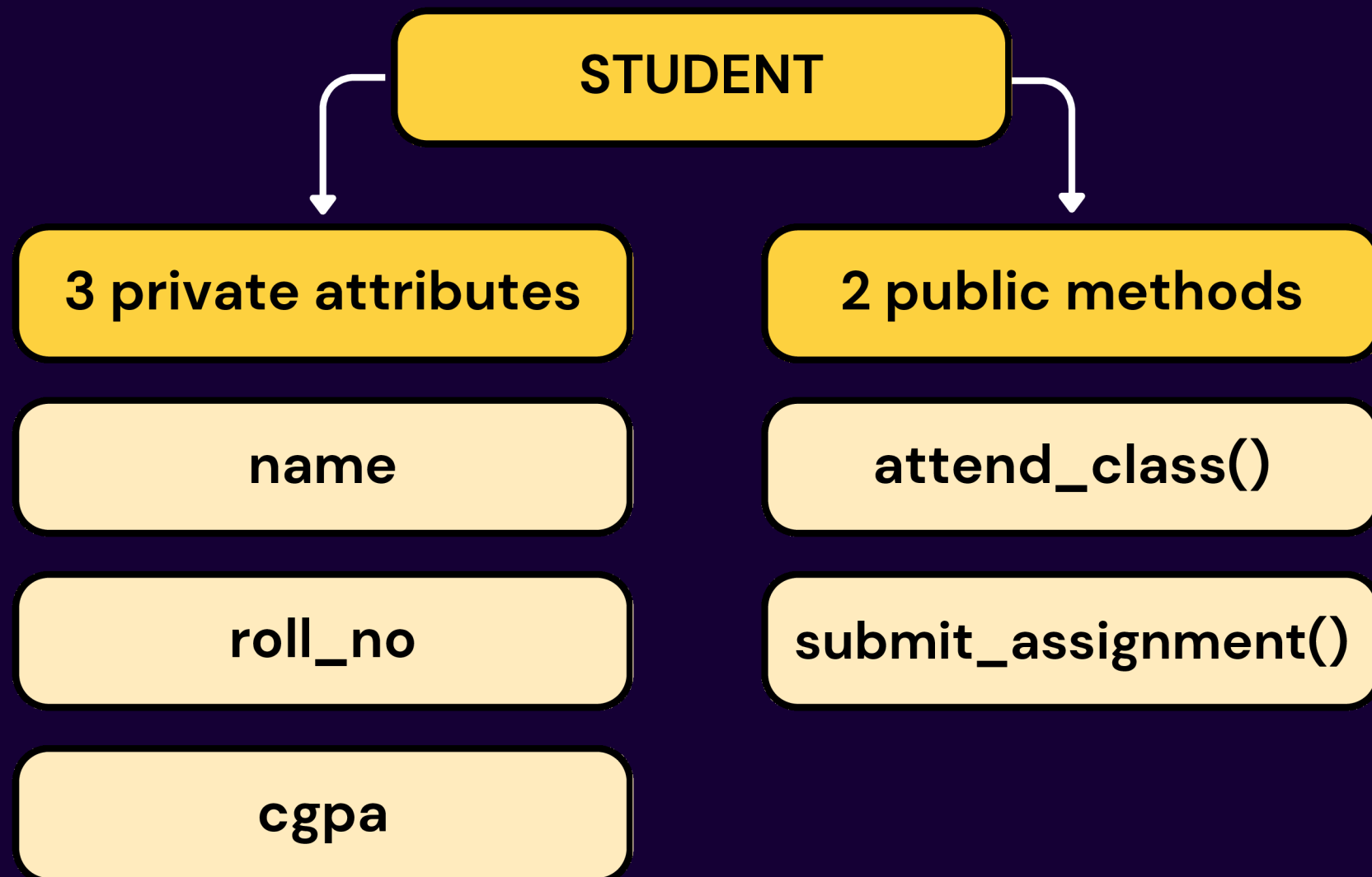
- Example: `class Dog(Animal):` – Dog is-a Animal.

6. Composition: A strong relationship where one object is made up of another object and cannot exist without it. Example: A Car has an Engine – if the Car is destroyed, the Engine is also destroyed.

7. Aggregation: A weaker relationship where one object contains another, but both can exist independently. Example: A Classroom has Students – if the Classroom closes, Students still exist.

Class Diagrams

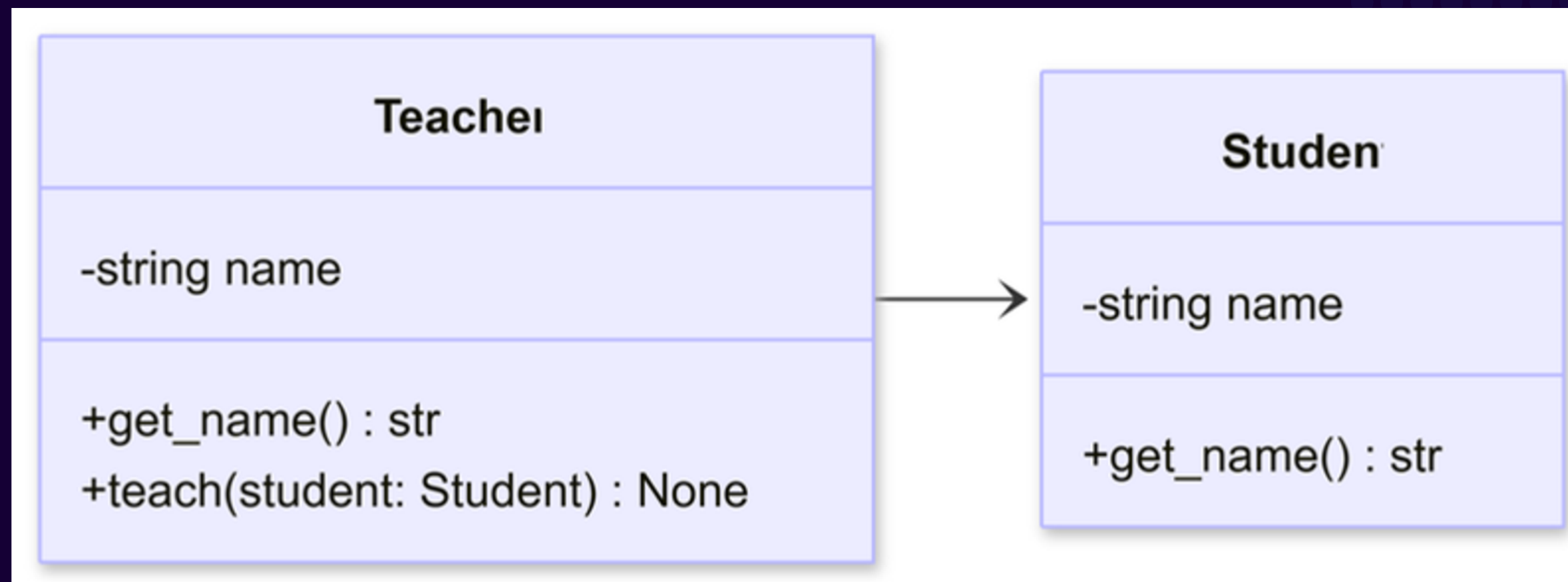
Shows the blueprint of your system like what **classes** you have, their properties (**attributes**), their actions (**methods**), and how these **classes** are **connected** to each other (like inheritance, association, etc.).



Association

Association is a connection between two or more classes that shows how their objects are related to each other. When one class's object uses or interacts with another class's object, that's association.

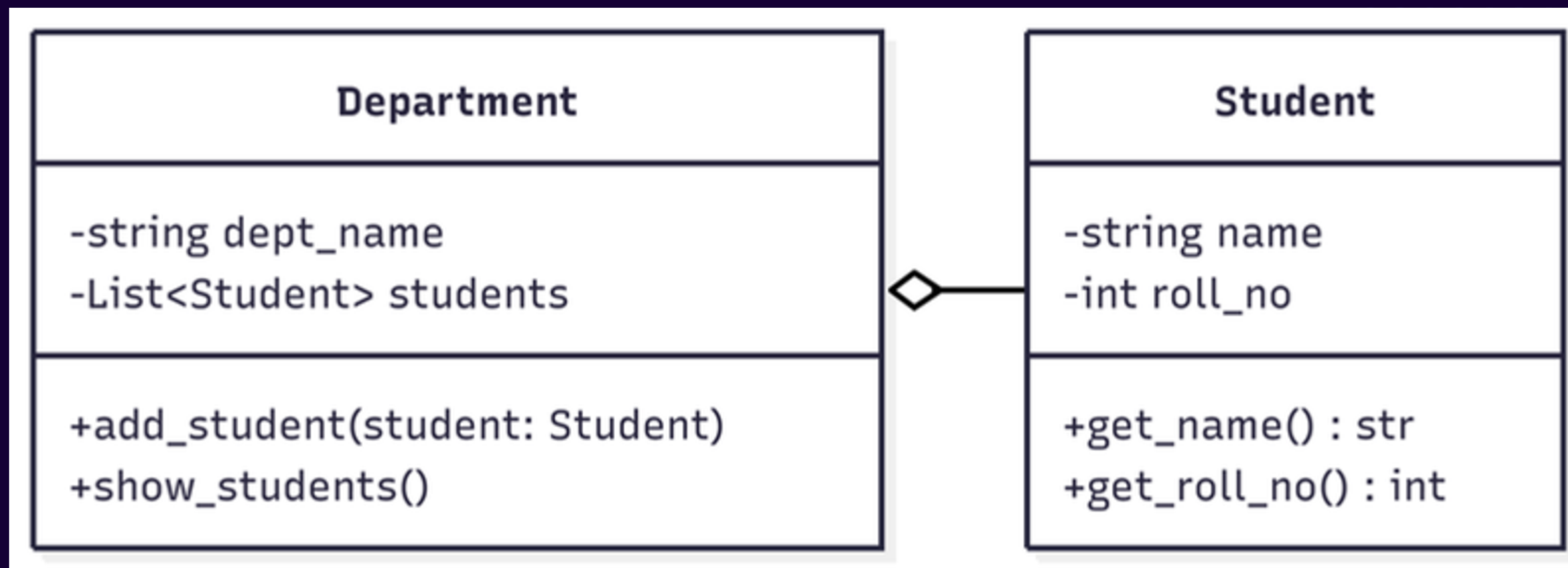
Simple terms: "Class A knows about Class B and can use it"



Aggregation

Aggregation is a weak "has-a" relationship where one class contains objects of another class, BUT the contained objects can survive independently even if the container is destroyed.

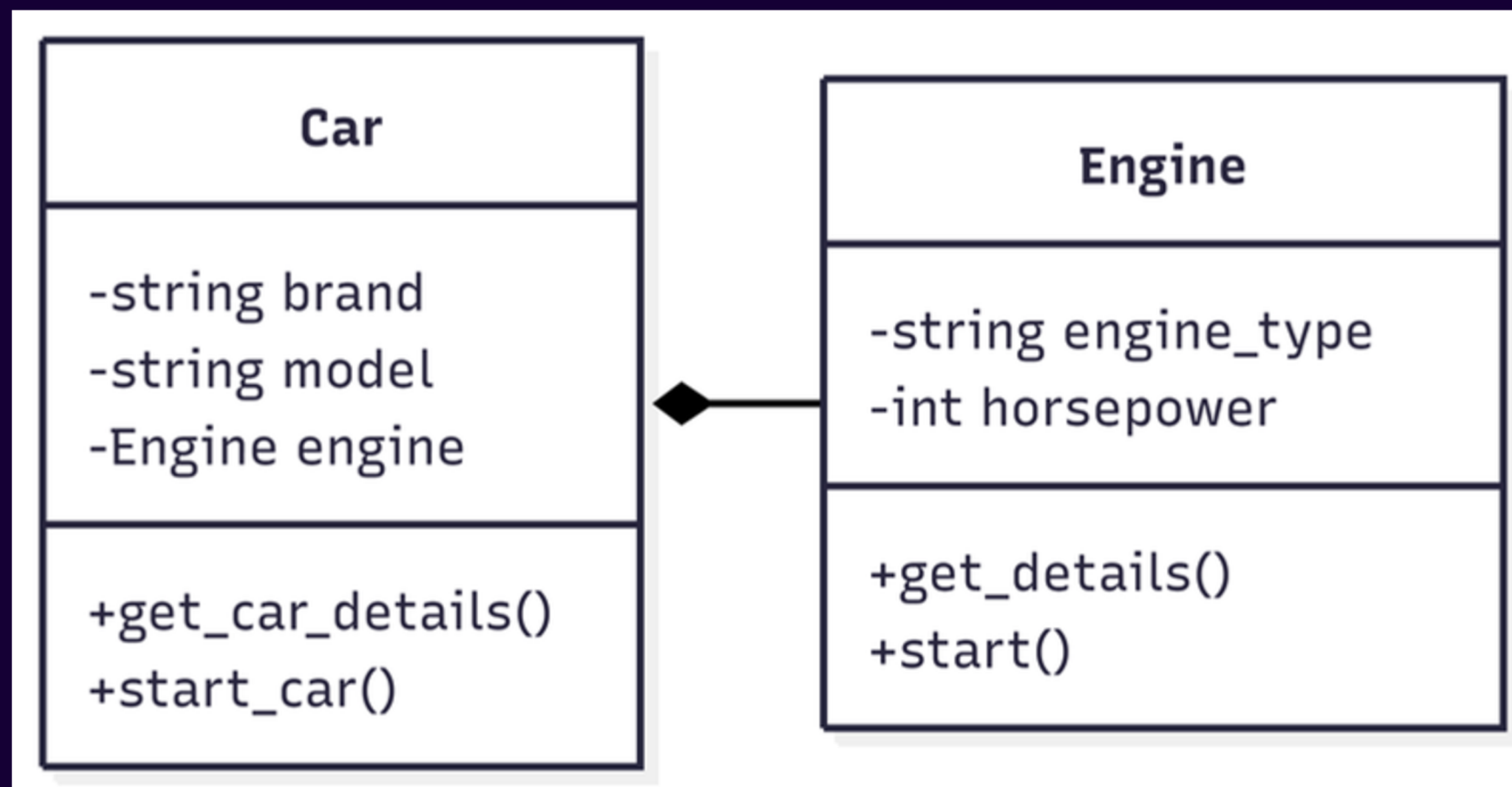
Simple terms: "Class A has Class B, but B can exist without A"



Composition

Composition is a strong "**has-a**" relationship where one class owns objects of another class. If the container object is destroyed, the contained objects are also destroyed automatically.

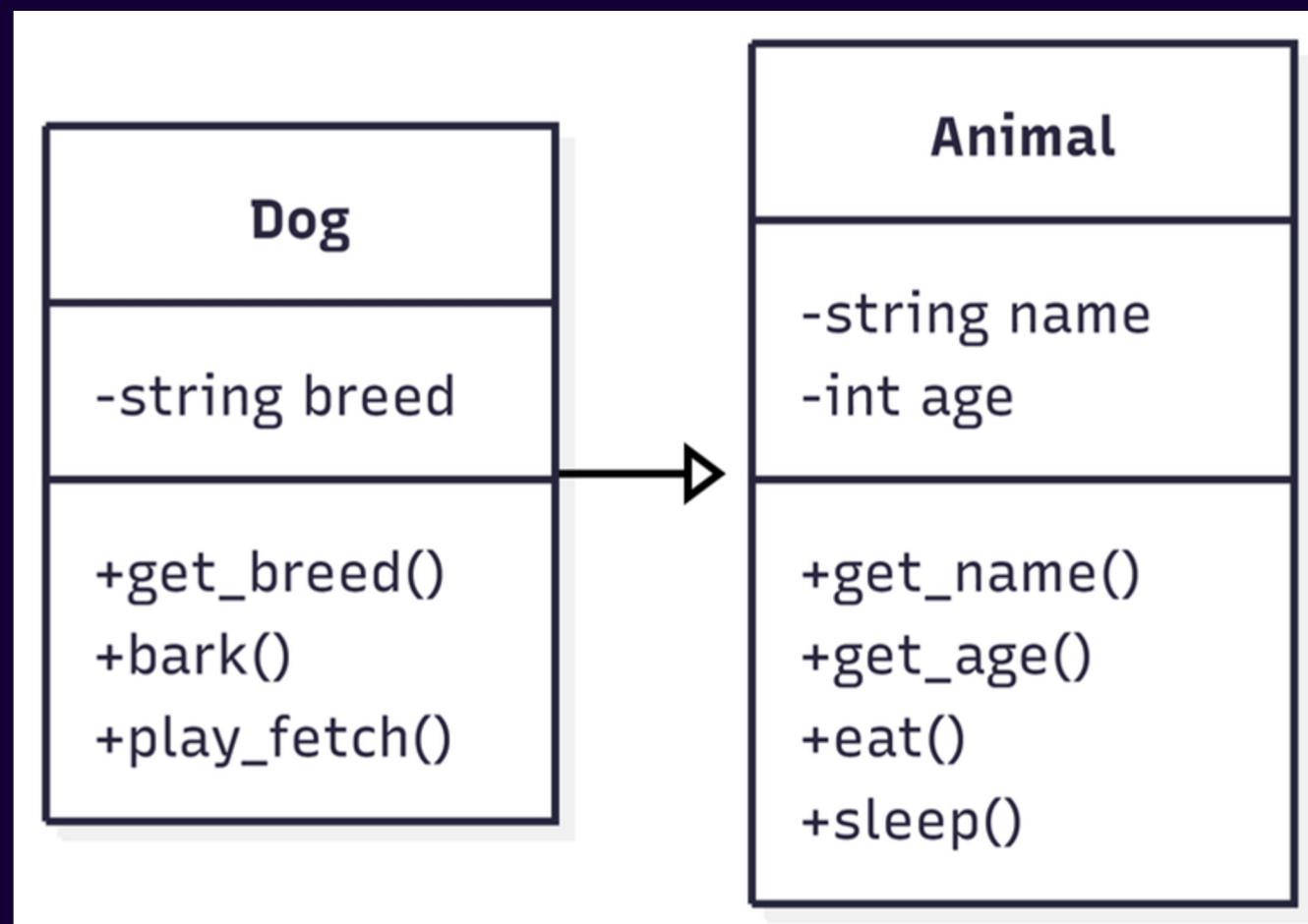
Simple terms: "Class A owns Class B completely, if A dies, B also dies"



Inheritance

Inheritance defines an **"is-a"** relationship where a **child class** (subclass) **inherits** properties and behaviors (methods) from a **parent class** (superclass).

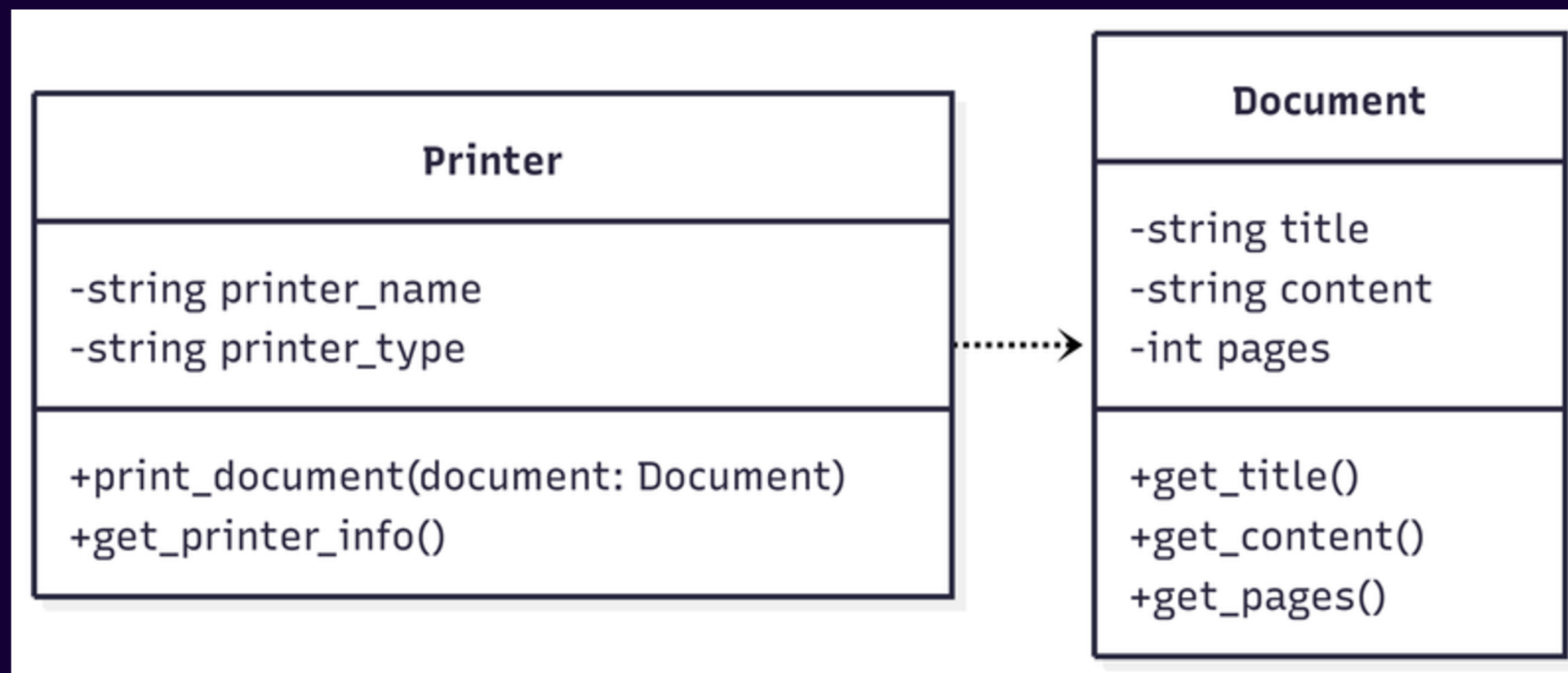
Simple terms: "Child class IS-A type of Parent class" – Child gets all features of Parent automatically.



Dependency

Dependency is a weak relationship where one class **temporarily uses** or relies on another class. The dependency is usually through method **parameters**, **return types**, or **temporary usage** – not permanent ownership.

Simple terms: "Class A uses Class B for a short time to do some work, then forgets about it"



Summary

- **Association:** Two classes are connected and their objects interact with each other.
- **Aggregation:** Weak "has-a" – Container has objects, but objects can survive independently.
- **Composition:** Strong "has-a" – Container owns objects, if container dies, objects die too.
- **Inheritance:** "is-a" relationship – Child class gets all features from Parent class.
- **Dependency:** One class temporarily uses another class to do its work.
- **Realization:** A class implements the methods defined by an interface (contract).